```
from google.colab import drive
import os
import shutil
from pathlib import Path

drive.mount('/content/drive')

PROJECT_ROOT = "/content/drive/MyDrive/pix2pix_cufs_project"
DATASET_DIR = os.path.join(PROJECT_ROOT, "dataset")
CHECKPOINT_DIR = os.path.join(PROJECT_ROOT, "checkpoints")
BEST_MODEL_DIR = os.path.join(PROJECT_ROOT, "best_model")
LOGS_DIR = os.path.join(PROJECT_ROOT, "logs")
SAMPLE_OUTPUTS = os.path.join(PROJECT_ROOT, "sample_outputs")
FINAL_MODEL_DIR = os.path.join(PROJECT_ROOT, "final_model")

for dir_path in [PROJECT_ROOT, DATASET_DIR, CHECKPOINT_DIR, BEST_MODEL_DIR,
                 LOGS_DIR, SAMPLE_OUTPUTS, FINAL_MODEL_DIR]:
    os.makedirs(dir_path, exist_ok=True)
    print(f"✓ Created: {dir_path}")

print(" Project structure initialized successfully!")
```

```
Mounted at /content/drive
✓ Created: /content/drive/MyDrive/pix2pix_cufs_project
✓ Created: /content/drive/MyDrive/pix2pix_cufs_project/dataset
✓ Created: /content/drive/MyDrive/pix2pix_cufs_project/checkpoints
✓ Created: /content/drive/MyDrive/pix2pix_cufs_project/best_model
✓ Created: /content/drive/MyDrive/pix2pix_cufs_project/logs
✓ Created: /content/drive/MyDrive/pix2pix_cufs_project/sample_outputs
✓ Created: /content/drive/MyDrive/pix2pix_cufs_project/final_model
 Project structure initialized successfully!
```

```
import zipfile
import os

ZIP_PATH = "/content/drive/MyDrive/face sketch and real face dataset.zip"
EXTRACT_PATH = DATASET_DIR

def extract_dataset():
    if not os.path.exists(ZIP_PATH):
        raise FileNotFoundError(f"Dataset not found at: {ZIP_PATH}")
    if not os.listdir(EXTRACT_PATH):
        with zipfile.ZipFile(ZIP_PATH, 'r') as zip_ref:
            zip_ref.extractall(EXTRACT_PATH)

def normalize_sketch_name(filename):
    name = os.path.splitext(filename)[0].lower()
    name = name.replace("f2-", "f-")
    name = name.replace("-sz1", "")
    return name

def normalize_photo_name(filename):
    return os.path.splitext(filename)[0].lower()

def verify_structure():
    sketch_dir = os.path.join(EXTRACT_PATH, "sketches")
    photo_dir = os.path.join(EXTRACT_PATH, "photos")

    sketch_files = os.listdir(sketch_dir)
    photo_files = os.listdir(photo_dir)

    sketch_map = {}
    for f in sketch_files:
        key = normalize_sketch_name(f)
        sketch_map[key] = f

    photo_map = {}
    for f in photo_files:
        key = normalize_photo_name(f)
        photo_map[key] = f

    common_keys = sorted(list(set(sketch_map.keys()).intersection(photo_map.keys())))

    return sketch_dir, photo_dir, common_keys, sketch_map, photo_map

extract_dataset()
SKETCH_DIR, PHOTO_DIR, COMMON_KEYS, SKETCH_MAP, PHOTO_MAP = verify_structure()
```

```
print("Total paired samples:", len(COMMON_KEYS))
```

```
Total paired samples: 134
```

```
!pip install lpips
```

```
Collecting lpips
  Downloading lpips-0.1.4-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: torch>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from lpips) (2.9.0+cu128)
Requirement already satisfied: torchvision>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from lpips) (0.24.0+cu128)
Requirement already satisfied: numpy>=1.14.3 in /usr/local/lib/python3.12/dist-packages (from lpips) (2.0.2)
Requirement already satisfied: scipy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from lpips) (1.16.3)
Requirement already satisfied: tqdm>=4.28.1 in /usr/local/lib/python3.12/dist-packages (from lpips) (4.67.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpips) (3.21.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpip
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpips) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpips) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpips) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpips) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpips) (2025.3.0
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->l
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->l
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->l
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpips
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lp
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpip
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0-
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->l
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch>=0.4.0->lpips) (3.5.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.2.1->lp
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=0.4
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=0.4.0->lpips)
Downloading lpips-0.1.4-py3-none-any.whl (53 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 53.8/53.8 kB 4.1 MB/s eta 0:00:00
Installing collected packages: lpips
Successfully installed lpips-0.1.4
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
import torchvision.models as models
from torch.cuda.amp import autocast, GradScaler

import numpy as np
import cv2
from PIL import Image
import random
from tqdm import tqdm
import json
from datetime import datetime
import matplotlib.pyplot as plt
from scipy import linalg
from skimage.metrics import structural_similarity as ssim
import lpips

torch.manual_seed(42)
np.random.seed(42)
random.seed(42)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print("Device:", device)

class Config:
    IMAGE_SIZE = 256
    INPUT_CHANNELS = 1
    OUTPUT_CHANNELS = 3

    BATCH_SIZE = 4
    NUM_EPOCHS = 250
    LEARNING_RATE = 2e-4
    BETA1 = 0.5
    BETA2 = 0.999
    L1_LAMBDA = 100
```

```
    PERCEPTUAL_LAMBDA = 10

    NGF = 64
    NDF = 64

    SAVE_EVERY = 10

    PHASE1_EPOCHS = 100
    PHASE2_EPOCHS = 100
    PHASE3_EPOCHS = 50

    FLIP_PROB = 0.5
    ROTATION_DEGREES = 5

config = Config()
```

```
Device: cuda
```

```
class UNetDown(nn.Module):
    def __init__(self, in_channels, out_channels, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_channels, out_channels, 4, 2, 1, bias=False)]
        if normalize:
            layers.append(nn.InstanceNorm2d(out_channels))
        layers.append(nn.LeakyReLU(0.2, inplace=True))
        if dropout:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)


class UNetUp(nn.Module):
    def __init__(self, in_channels, out_channels, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_channels, out_channels, 4, 2, 1, bias=False),
            nn.InstanceNorm2d(out_channels),
            nn.ReLU(inplace=True)
        ]
        if dropout:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)

    def forward(self, x, skip_input):
        x = self.model(x)
        x = torch.cat((x, skip_input), 1)
        return x


class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=3):
        super().__init__()

        self.down1 = UNetDown(in_channels, config.NGF, normalize=False)
        self.down2 = UNetDown(config.NGF, config.NGF*2)
        self.down3 = UNetDown(config.NGF*2, config.NGF*4)
        self.down4 = UNetDown(config.NGF*4, config.NGF*8)
        self.down5 = UNetDown(config.NGF*8, config.NGF*8)
        self.down6 = UNetDown(config.NGF*8, config.NGF*8)
        self.down7 = UNetDown(config.NGF*8, config.NGF*8)
        self.down8 = UNetDown(config.NGF*8, config.NGF*8, normalize=False)

        self.up1 = UNetUp(config.NGF*8, config.NGF*8, dropout=0.5)
        self.up2 = UNetUp(config.NGF*16, config.NGF*8, dropout=0.5)
        self.up3 = UNetUp(config.NGF*16, config.NGF*8, dropout=0.5)
        self.up4 = UNetUp(config.NGF*16, config.NGF*8)
        self.up5 = UNetUp(config.NGF*16, config.NGF*4)
        self.up6 = UNetUp(config.NGF*8, config.NGF*2)
        self.up7 = UNetUp(config.NGF*4, config.NGF)

        self.final = nn.Sequential(
            nn.ConvTranspose2d(config.NGF*2, out_channels, 4, 2, 1),
            nn.Tanh()
        )

    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)
        d3 = self.down3(d2)
```

```
        d4 = self.down4(d3)
        d5 = self.down5(d4)
        d6 = self.down6(d5)
        d7 = self.down7(d6)
        d8 = self.down8(d7)

        u1 = self.up1(d8, d7)
        u2 = self.up2(u1, d6)
        u3 = self.up3(u2, d5)
        u4 = self.up4(u3, d4)
        u5 = self.up5(u4, d3)
        u6 = self.up6(u5, d2)
        u7 = self.up7(u6, d1)

        return self.final(u7)

generator = GeneratorUNet().to(device)
```

```
class Discriminator(nn.Module):
    def __init__(self, in_channels=4):
        super().__init__()

        def block(in_f, out_f, normalize=True):
            layers = [nn.Conv2d(in_f, out_f, 4, 2, 1)]
            if normalize:
                layers.append(nn.InstanceNorm2d(out_f))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            *block(in_channels, config.NDF, normalize=False),
            *block(config.NDF, config.NDF*2),
            *block(config.NDF*2, config.NDF*4),
            *block(config.NDF*4, config.NDF*8),
            nn.Conv2d(config.NDF*8, 1, 4, padding=1)
        )

    def forward(self, sketch, photo):
        x = torch.cat((sketch, photo), 1)
        return self.model(x)

discriminator = Discriminator().to(device)
```

```
class PerceptualLoss(nn.Module):
    def __init__(self):
        super().__init__()
        vgg = models.vgg19(pretrained=True).features[:18].eval()
        for p in vgg.parameters():
            p.requires_grad = False
        self.vgg = vgg.to(device)

    def forward(self, gen, target):
        mean = torch.tensor([0.485,0.456,0.406]).view(1,3,1,1).to(device)
        std = torch.tensor([0.229,0.224,0.225]).view(1,3,1,1).to(device)

        gen = ((gen+1)/2 - mean)/std
        target = ((target+1)/2 - mean)/std

        return torch.mean(torch.abs(self.vgg(gen)-self.vgg(target)))

perceptual_loss_fn = PerceptualLoss()
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth" to /root/.cache/torch/hub/checkpoints/vgg19-dcbb9e9d.p
100%|██████████| 548M/548M [00:07<00:00, 78.9MB/s]
```

```
class SketchPhotoDataset(Dataset):
    def __init__(self, sketch_dir, photo_dir, keys, sketch_map, photo_map, transform=True):
        self.sketch_dir = sketch_dir
        self.photo_dir = photo_dir
        self.keys = keys
        self.sketch_map = sketch_map
        self.photo_map = photo_map
        self.transform = transform
```

```
    def __len__(self):
        return len(self.keys)

    def __getitem__(self, idx):
        key = self.keys[idx]

        sketch = Image.open(os.path.join(self.sketch_dir, self.sketch_map[key])).convert('L')
        photo = Image.open(os.path.join(self.photo_dir, self.photo_map[key])).convert('RGB')

        sketch = sketch.resize((286, 286))
        photo = photo.resize((286, 286))

        if random.random() < config.FLIP_PROB:
            sketch = transforms.functional.hflip(sketch)
            photo = transforms.functional.hflip(photo)

        if random.random() < 0.3:
            angle = random.uniform(-config.ROTATION_DEGREES, config.ROTATION_DEGREES)
            sketch = transforms.functional.rotate(sketch, angle)
            photo = transforms.functional.rotate(photo, angle)

        i, j, h, w = transforms.RandomCrop.get_params(sketch, output_size=(config.IMAGE_SIZE, config.IMAGE_SIZE))
        sketch = transforms.functional.crop(sketch, i, j, h, w)
        photo = transforms.functional.crop(photo, i, j, h, w)

        sketch = transforms.ToTensor()(sketch)
        sketch = transforms.Normalize([0.5], [0.5])(sketch)

        photo = transforms.ToTensor()(photo)
        photo = transforms.Normalize([0.5,0.5,0.5],[0.5,0.5,0.5])(photo)

        return {'sketch': sketch, 'photo': photo}

dataset = SketchPhotoDataset(SKETCH_DIR, PHOTO_DIR, COMMON_KEYS, SKETCH_MAP, PHOTO_MAP)
dataloader = DataLoader(dataset, batch_size=config.BATCH_SIZE, shuffle=True, num_workers=2)
```

```
generator = GeneratorUNet().to(device)
discriminator = Discriminator().to(device)

criterion_GAN = nn.MSELoss()
criterion_L1 = nn.L1Loss()

optimizer_G = optim.Adam(generator.parameters(), lr=config.LEARNING_RATE, betas=(config.BETA1, config.BETA2))
optimizer_D = optim.Adam(discriminator.parameters(), lr=config.LEARNING_RATE, betas=(config.BETA1, config.BETA2))

scheduler_G = optim.lr_scheduler.MultiStepLR(
    optimizer_G,
    milestones=[config.PHASE1_EPOCHS, config.PHASE1_EPOCHS + config.PHASE2_EPOCHS],
    gamma=0.5
)

scheduler_D = optim.lr_scheduler.MultiStepLR(
    optimizer_D,
    milestones=[config.PHASE1_EPOCHS, config.PHASE1_EPOCHS + config.PHASE2_EPOCHS],
    gamma=0.5
)

def save_checkpoint(epoch, best_fid):
    checkpoint = {
        'epoch': epoch,
        'generator_state_dict': generator.state_dict(),
        'discriminator_state_dict': discriminator.state_dict(),
        'optimizer_G_state_dict': optimizer_G.state_dict(),
        'optimizer_D_state_dict': optimizer_D.state_dict(),
        'scheduler_G_state_dict': scheduler_G.state_dict(),
        'scheduler_D_state_dict': scheduler_D.state_dict(),
        'best_fid': best_fid
    }

    torch.save(checkpoint, os.path.join(CHECKPOINT_DIR, 'latest_checkpoint.pth'))

    if epoch % config.SAVE_EVERY == 0:
        torch.save(checkpoint, os.path.join(CHECKPOINT_DIR, f'checkpoint_epoch_{epoch}.pth'))

def load_checkpoint():
    checkpoints = [f for f in os.listdir(CHECKPOINT_DIR) if f.endswith('.pth')]

    if not checkpoints:
        print("No checkpoints found. Starting from scratch.")
        return 0, float('inf')
```

```
    checkpoints.sort()
    latest = checkpoints[-1]
    path = os.path.join(CHECKPOINT_DIR, latest)

    print("Loading:", latest)

    checkpoint = torch.load(path)
    generator.load_state_dict(checkpoint['generator_state_dict'])
    discriminator.load_state_dict(checkpoint['discriminator_state_dict'])
    optimizer_G.load_state_dict(checkpoint['optimizer_G_state_dict'])
    optimizer_D.load_state_dict(checkpoint['optimizer_D_state_dict'])
    scheduler_G.load_state_dict(checkpoint['scheduler_G_state_dict'])
    scheduler_D.load_state_dict(checkpoint['scheduler_D_state_dict'])

    print("Resumed from epoch", checkpoint['epoch'])
    return checkpoint['epoch'] + 1, checkpoint['best_fid']
```

```
START_EPOCH, BEST_FID = load_checkpoint()
```

```
Loading: latest_checkpoint.pth
Resumed from epoch 109
```

```
def train():
    global BEST_FID

    history = {'G_loss': [], 'D_loss': []}

    print("Starting training from epoch:", START_EPOCH)

    for epoch in range(START_EPOCH, config.NUM_EPOCHS):

        print("Running Epoch:", epoch)

        generator.train()
        discriminator.train()

        epoch_G_loss = 0
        epoch_D_loss = 0

        for batch in dataloader:

            real_sketch = batch['sketch'].to(device)
            real_photo = batch['photo'].to(device)

            pred_shape = discriminator(real_sketch, real_photo).shape
            real_label = torch.ones(pred_shape).to(device) * 0.9
            fake_label = torch.zeros(pred_shape).to(device)

            optimizer_D.zero_grad()

            fake_photo = generator(real_sketch)

            pred_real = discriminator(real_sketch, real_photo)
            loss_D_real = criterion_GAN(pred_real, real_label)

            pred_fake = discriminator(real_sketch, fake_photo.detach())
            loss_D_fake = criterion_GAN(pred_fake, fake_label)

            loss_D = 0.5 * (loss_D_real + loss_D_fake)
            loss_D.backward()
            optimizer_D.step()

            optimizer_G.zero_grad()

            pred_fake = discriminator(real_sketch, fake_photo)
            loss_G_GAN = criterion_GAN(pred_fake, real_label)
            loss_G_L1 = criterion_L1(fake_photo, real_photo) * config.L1_LAMBDA

            loss_G = loss_G_GAN + loss_G_L1
            loss_G.backward()
            optimizer_G.step()

            epoch_G_loss += loss_G.item()
            epoch_D_loss += loss_D.item()

        scheduler_G.step()
        scheduler_D.step()
```

```
            avg_G_loss = epoch_G_loss / len(dataloader)
            avg_D_loss = epoch_D_loss / len(dataloader)

            history['G_loss'].append(avg_G_loss)
            history['D_loss'].append(avg_D_loss)

            save_checkpoint(epoch, BEST_FID)

            generator.eval()
            with torch.no_grad():
                sample_batch = next(iter(dataloader))
                sketch = sample_batch['sketch'][:4].to(device)
                fake = generator(sketch).cpu()

            fig, axes = plt.subplots(4, 2, figsize=(8, 12))
            for i in range(4):
                axes[i, 0].imshow((sketch[i].cpu().squeeze().numpy() + 1) / 2, cmap='gray')
                axes[i, 0].axis('off')
                axes[i, 1].imshow(((fake[i].numpy() + 1) / 2).transpose(1, 2, 0))
                axes[i, 1].axis('off')

            plt.tight_layout()
            plt.savefig(os.path.join(SAMPLE_OUTPUTS, f'epoch_{epoch:03d}.png'))
            plt.close()

            generator.train()

            print("Epoch:", epoch)
            print("G Loss:", avg_G_loss)
            print("D Loss:", avg_D_loss)
            print("--------------------------------------------------")

        return history


    START_EPOCH, BEST_FID = load_checkpoint()
    history = train()
```

```
G Loss: 10.864047555362477
D Loss: 0.07138976728653207
--------------------------------------------------
Running Epoch: 181
Epoch: 181
G Loss: 10.749931363498463
D Loss: 0.07734567628187292
--------------------------------------------------
Running Epoch: 182
Epoch: 182
G Loss: 10.703961316276999
D Loss: 0.08030442772980999
```

```
!pip install onnx onnxscript
```

```
Collecting onnx
  Downloading onnx-1.20.1-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Collecting onnxscript
  Downloading onnxscript-0.6.2-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.12/dist-packages (from onnx) (2.0.2)
Requirement already satisfied: protobuf>=4.25.1 in /usr/local/lib/python3.12/dist-packages (from onnx) (5.29.6)
Requirement already satisfied: typing_extensions>=4.7.1 in /usr/local/lib/python3.12/dist-packages (from onnx) (4.15.0)
Requirement already satisfied: ml_dtypes>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from onnx) (0.5.4)
Collecting onnx_ir<2,>=0.1.15 (from onnxscript)
  Downloading onnx_ir-0.1.16-py3-none-any.whl.metadata (3.2 kB)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from onnxscript) (26.0)
Downloading onnx-1.20.1-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (17.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.5/17.5 MB 95.7 MB/s eta 0:00:00
Downloading onnxscript-0.6.2-py3-none-any.whl (689 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 689.1/689.1 kB 57.9 MB/s eta 0:00:00
Downloading onnx_ir-0.1.16-py3-none-any.whl (159 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 159.3/159.3 kB 19.1 MB/s eta 0:00:00
Installing collected packages: onnx, onnx_ir, onnxscript
Successfully installed onnx-1.20.1 onnx_ir-0.1.16 onnxscript-0.6.2
```

```
def export_final_model():
    final_path = os.path.join(FINAL_MODEL_DIR, 'generator_final.pth')
    torch.save(generator.state_dict(), final_path)

    generator.eval()
    example_input = torch.randn(1, 1, config.IMAGE_SIZE, config.IMAGE_SIZE).to(device)

    traced_model = torch.jit.trace(generator, example_input)
    traced_model.save(os.path.join(FINAL_MODEL_DIR, 'generator_traced.pt'))

    torch.onnx.export(
        generator,
        example_input,
        os.path.join(FINAL_MODEL_DIR, 'generator.onnx'),
        input_names=['sketch'],
        output_names=['photo'],
        dynamic_axes={'sketch': {0: 'batch_size'}, 'photo': {0: 'batch_size'}}
    )

    print("Final model exported")

export_final_model()
```

```
/tmp/ipython-input-3181686186.py:11: UserWarning: # 'dynamic_axes' is not recommended when dynamo=True, and may lead to 'tor
  torch.onnx.export(
W0219 16:05:18.611000 914 torch/onnx/_internal/exporter/_schemas.py:455] Missing annotation for parameter 'input' from (inpu
W0219 16:05:18.612000 914 torch/onnx/_internal/exporter/_schemas.py:455] Missing annotation for parameter 'boxes' from (inpu
W0219 16:05:18.615000 914 torch/onnx/_internal/exporter/_schemas.py:455] Missing annotation for parameter 'input' from (inpu
W0219 16:05:18.617000 914 torch/onnx/_internal/exporter/_schemas.py:455] Missing annotation for parameter 'boxes' from (inpu
[torch.onnx] Obtain model graph for `GeneratorUNet([...]` with `torch.export.export(..., strict=False)`...
[torch.onnx] Obtain model graph for `GeneratorUNet([...]` with `torch.export.export(..., strict=False)`... ✅
[torch.onnx] Run decomposition...
[torch.onnx] Run decomposition... ✅
[torch.onnx] Translate the graph into ONNX...
[torch.onnx] Translate the graph into ONNX... ✅
Applied 8 of general pattern rewrite rules.
Final model exported
```

```
!pip install lpips

import lpips

lpips_fn = lpips.LPIPS(net='alex').to(device)
lpips_fn.eval()
```

```
          (0): Dropout(p=0.5, inplace=False)
          (1): Conv2d(64, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
        )
      )
      (lin1): NetLinLayer(
        (model): Sequential(
          (0): Dropout(p=0.5, inplace=False)
          (1): Conv2d(192, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
        )
      )
      (lin2): NetLinLayer(
        (model): Sequential(
          (0): Dropout(p=0.5, inplace=False)
          (1): Conv2d(384, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
        )
      )
      (lin3): NetLinLayer(
        (model): Sequential(
          (0): Dropout(p=0.5, inplace=False)
          (1): Conv2d(256, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
        )
      )
      (lin4): NetLinLayer(
        (model): Sequential(
          (0): Dropout(p=0.5, inplace=False)
          (1): Conv2d(256, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
        )
      )
      (lins): ModuleList(
        (0): NetLinLayer(
          (model): Sequential(
            (0): Dropout(p=0.5, inplace=False)
            (1): Conv2d(64, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
          )
        )
        (1): NetLinLayer(
          (model): Sequential(
            (0): Dropout(p=0.5, inplace=False)
            (1): Conv2d(192, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
          )
        )
        (2): NetLinLayer(
          (model): Sequential(
            (0): Dropout(p=0.5, inplace=False)
            (1): Conv2d(384, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
          )
        )
        (3-4): 2 x NetLinLayer(
          (model): Sequential(
            (0): Dropout(p=0.5, inplace=False)
            (1): Conv2d(256, 1, kernel_size=(1, 1), stride=(1, 1), bias=False)
          )
        )
      )
    )
```

```
print(generator)
```

```
    )
  )
  (up5): UNetUp(
    (model): Sequential(
      (0): ConvTranspose2d(1024, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
      (1): InstanceNorm2d(256, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
      (2): ReLU(inplace=True)
    )
  )
  (up6): UNetUp(
    (model): Sequential(
      (0): ConvTranspose2d(512, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
      (1): InstanceNorm2d(128, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
      (2): ReLU(inplace=True)
    )
  )
  (up7): UNetUp(
    (model): Sequential(
      (0): ConvTranspose2d(256, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
      (1): InstanceNorm2d(64, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
      (2): ReLU(inplace=True)
    )
  )
  (final): Sequential(
    (0): ConvTranspose2d(128, 3, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1))
    (1): Tanh()
  )
)
```

```
class FIDScore:
    def __init__(self):
        self.inception = models.inception_v3(pretrained=True, transform_input=False)
        self.inception.fc = nn.Identity()
        self.inception.eval().to(device)
        for param in self.inception.parameters():
            param.requires_grad = False

    def get_features(self, images):
        if images.shape[2] != 299:
            images = nn.functional.interpolate(images, size=(299, 299), mode='bilinear', align_corners=False)

        mean = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1).to(device)
        std = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1).to(device)

        images = (images * 0.5 + 0.5 - mean) / std

        with torch.no_grad():
            features = self.inception(images)

        return features.cpu().numpy()

    def calculate_fid(self, real_features, fake_features):
        mu1, sigma1 = real_features.mean(axis=0), np.cov(real_features, rowvar=False)
        mu2, sigma2 = fake_features.mean(axis=0), np.cov(fake_features, rowvar=False)

        ssdiff = np.sum((mu1 - mu2) ** 2)
        covmean = linalg.sqrtm(sigma1.dot(sigma2))

        if np.iscomplexobj(covmean):
            covmean = covmean.real

        fid = ssdiff + np.trace(sigma1 + sigma2 - 2 * covmean)
        return fid

fid_calculator = FIDScore()
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/inception_v3_google-0cc3c7bd.pth" to /root/.cache/torch/hub/checkpoints/in
100%|██████████| 104M/104M [00:01<00:00, 58.0MB/s]
```

```
fid_calculator = FIDScore()
```

```
def evaluate_model():
    generator.eval()

    all_ssim = []
    all_lpips = []

    with torch.no_grad():
        for batch in dataloader:
            sketch = batch['sketch'].to(device)
```

```
            real = batch['photo'].to(device)
            fake = generator(sketch)

            for i in range(real.size(0)):
                real_np = ((real[i].cpu().numpy() + 1) / 2).transpose(1, 2, 0)
                fake_np = ((fake[i].cpu().numpy() + 1) / 2).transpose(1, 2, 0)
                ssim_score = ssim(real_np, fake_np, data_range=1.0, channel_axis=2)
                all_ssim.append(ssim_score)

            lpips_score = lpips_fn(real, fake).squeeze().cpu().numpy()
            if np.isscalar(lpips_score):
                all_lpips.append(lpips_score)
            else:
                all_lpips.extend(lpips_score.tolist())

    real_features = []
    fake_features = []

    with torch.no_grad():
        for batch in dataloader:
            sketch = batch['sketch'].to(device)
            real = batch['photo'].to(device)
            fake = generator(sketch)

            real_features.append(fid_calculator.get_features(real))
            fake_features.append(fid_calculator.get_features(fake))

    real_features = np.concatenate(real_features)
    fake_features = np.concatenate(fake_features)

    fid_score = fid_calculator.calculate_fid(real_features, fake_features)

    print("FID:", fid_score)
    print("SSIM:", np.mean(all_ssim))
    print("LPIPS:", np.mean(all_lpips))

    results = {
        'fid': float(fid_score),
        'ssim_mean': float(np.mean(all_ssim)),
        'lpips_mean': float(np.mean(all_lpips)),
        'num_samples': len(dataset)
    }

    with open(os.path.join(FINAL_MODEL_DIR, 'evaluation_results.json'), 'w') as f:
        json.dump(results, f)

    return results

evaluate_model()
```

```
FID: 92.07250116796219
SSIM: 0.6799669
LPIPS: 0.17129689099183723
{'fid': 92.07250116796219,
 'ssim_mean': 0.679966926574707,
 'lpips_mean': 0.17129689099183723,
 'num_samples': 134}
```

```
from google.colab import files
import matplotlib.pyplot as plt
from PIL import Image
import torchvision.transforms as transforms
import torch
import numpy as np

generator.eval()

uploaded = files.upload()

for filename in uploaded.keys():
    sketch = Image.open(filename).convert('L')
    sketch = sketch.resize((config.IMAGE_SIZE, config.IMAGE_SIZE))

    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize([0.5], [0.5])
    ])

    sketch_tensor = transform(sketch).unsqueeze(0).to(device)

    with torch.no_grad():
        fake_photo = generator(sketch_tensor)
```